Ministerul Educației și Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică și Microelectronică

Departamentul Ingineria Software și Automatică

## Laboratory Work 1

# Introduction to Linux and Operating System Resources

Operating Systems

**Done by:** st. gr. FAF-241, Cațț Chiril

**Checked by:**
asist. univ. Reițman Patricia

Chișinău – 2026

# Contents

**1 Introduction** **1**

1.1 Objective . . . . . 1

1.2 Tasks . . . . . 1

1.3 Environment and Method . . . . . 1

**2 Filesystem and File Permissions** **2**

2.1 Inspecting the Home Directory . . . . . 2

2.2 Changing Permissions . . . . . 2

2.3 Purpose of Important Directories . . . . . 3

**3 Processes and Process States** **3**

3.1 Creating a Background Process . . . . . 3

3.2 Process Listing and PID 1 . . . . . 4

3.3 Reading Process Information from proc . . . . . 4

**4 Memory Management** **6**

4.1 System Memory and Swap . . . . . 6

4.2 Memory Used by the sleep Process . . . . . 6

**5 Storage, Mounts, and Devices** **7**

5.1 Block Devices . . . . . 7

5.2 The Root Filesystem . . . . . 7

5.3 Device Nodes . . . . . 8

**6 Discussion** **8**

**7 Conclusion** **9**

# 1 Introduction

An operating system manages the computer's hardware and provides abstractions that programs can use safely. Its main responsibilities include organizing files, controlling processes, allocating memory, managing persistent storage, and exposing input/output devices. Linux makes these responsibilities observable through ordinary command-line utilities and virtual filesystems such as `/proc` and `/dev`.

This laboratory work examines those facilities from inside a Docker container. The container provides an isolated Linux userspace while sharing the host's kernel. Consequently, the commands show the container's process namespace, filesystem view, and assigned resources rather than a complete view of the macOS host.

## 1.1 Objective

The objective is to become familiar with the Linux command line and to observe how the operating system represents and manages files, processes, memory, storage, mounted filesystems, and devices.

## 1.2 Tasks

The practical work was divided into four parts:

1. inspect the home directory and file metadata, then change file permissions;
2. create and inspect a background process and interpret its state;
3. examine system memory and the resident memory used by one process;
4. inspect block devices, the root mount, and device files.

## 1.3 Environment and Method

All commands were executed as the `root` user in a Docker container. Each command was run interactively, its output was captured, and the result was interpreted in relation to the resource being managed. Table 1 summarizes the principal tools used.

Table 1: Commands used during the laboratory work

| Command | Purpose |
|---|---|
| `ls, chmod` | Inspect directory entries and modify permission bits. |
| `jobs, ps` | Display shell jobs and processes visible in the container. |
| `/proc/PID/status` | Read kernel-maintained status and memory data for a process. |
| `free` | Summarize physical memory and swap usage. |
| `lsblk, findmnt` | Inspect block devices and mounted filesystems. |
| `ls /dev` | List device nodes exposed to the container. |

# 2 Filesystem and File Permissions

## 2.1 Inspecting the Home Directory

The command `ls -la ~` lists every entry in the home directory, including hidden files whose names begin with a dot. The long format reports the file type and permissions, link count, owner, group, size, modification time, and name. In the container, the home directory is `/root`; it contains the standard shell configuration files `.bashrc` and `.profile`.

```
root@b6f22478da98:/# ls -la ~
total 16
drwx------ 2 root root 4096 Sep 11 02:12 .
drwxr-xr-x 1 root root 4096 Sep 28 13:54 ..
-rw-r--r-- 1 root root 3106 Apr 22  2024 .bashrc
-rw-r--r-- 1 root root  161 Apr 22  2024 .profile
root@b6f22478da98:/#
```

Figure 2.1 – Long listing of the root user's home directory

The owner and group fields are both `root`, which is expected because the commands were executed by the container's root account. The first character of the mode identifies the object type: `d` denotes a directory and `-` denotes a regular file. The remaining nine characters form three permission groups for the owner, group, and others.

## 2.2 Changing Permissions

Before the permission change, `note.txt` had the mode `-rw-r--r--`. This means that the owner could read and write the file, while the group and all other users could only read it.




Figure 2.2 – Permissions of note.txt before applying chmod

The command `chmod 600 note.txt` assigns read and write permission to the owner and removes every permission from the group and others. The octal value is interpreted as (6=4+2) for owner read and write, followed by zero permissions for the other two classes.

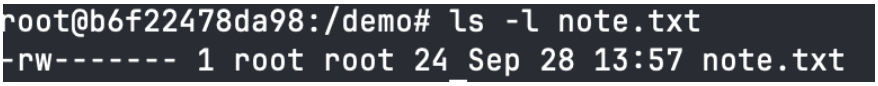



Figure 2.3 – Permissions of note.txt after applying chmod 600

After the change, the mode is `-rw----`. The initial dash still marks a regular file; `rw-` grants the owner read and write access, and the two `--` groups deny access to the group and others. This demonstrates how Linux enforces discretionary access control through per-file ownership and permission bits.

### 2.3 Purpose of Important Directories

Two important top-level directories are:

- `/etc`, which stores system-wide configuration files;
- `/var`, which stores variable data that changes while the system runs, including logs, caches, spool data, and some container-runtime storage.

## 3 Processes and Process States

### 3.1 Creating a Background Process

The command `sleep 300 &` starts `sleep` as a background job. The shell immediately returns a prompt and reports the job number and process identifier. The `jobs` command shows that the job is running, while `ps -ef | grep sleep` locates it in the process table.

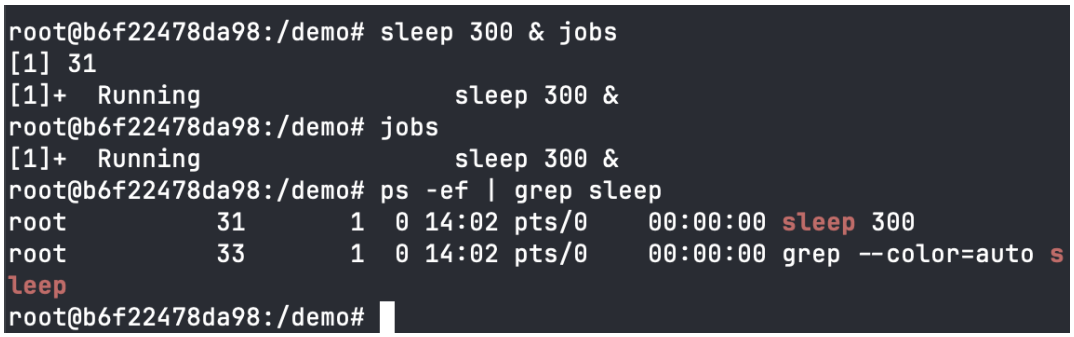



Figure 3.1 – Starting and locating a background sleep process

A background job is still an ordinary process. The ampersand only instructs the shell not to wait synchronously for its completion. This permits other commands to be entered while `sleep` remains active.

## 3.2 Process Listing and PID 1

The `ps aux` output includes the user, PID, CPU and memory percentages, virtual and resident memory, terminal, state, start time, accumulated CPU time, and command. In this container, PID 1 is `bash`, as shown in Figure 3.2. In a full Linux system PID 1 is normally an init system, but a container can launch its main application directly as PID 1 in its isolated PID namespace.

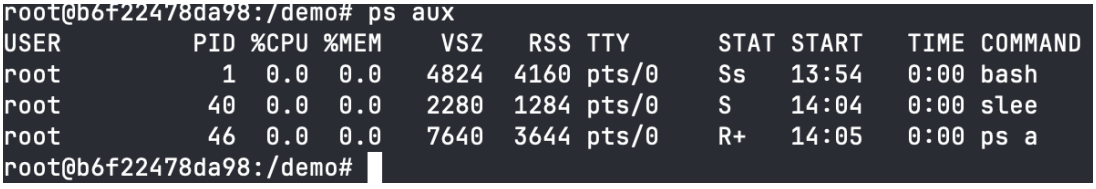



Figure 3.2 – Process list showing bash as PID 1

The command `ps aux | wc -l` returned 5. This value counts the header and the short-lived commands that form the pipeline as well as the persistent processes. It therefore describes the lines visible during that particular measurement, not a permanent process count.

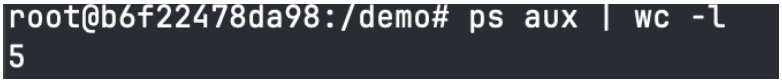



Figure 3.3 – Counting the lines produced by ps aux

## 3.3 Reading Process Information from proc

Linux exposes process information through `/proc`. After saving the process identifier in `PID`, the command `cat /proc/$PID/status | head -20` displays the beginning of

the kernel-maintained status record. The record identifies the command as `sleep`, gives PID 40 and parent PID 1, and reports its credentials and memory information.

```
root@b6f22478da98:/demo# cat /proc/$PID/status | head -20
Name:   sleep
Umask:  0022
State:  S (sleeping)
Tgid:   40
Ngid:   0
Pid:    40
PPid:   1
TracerPid:      0
Uid:    0       0       0       0
Gid:    0       0       0       0
FDSize: 256
Groups: 0
NStgid: 40
NSpid:  40
NSpgid: 40
NSsid:  1
Kthread:        0
VmPeak:     2280 kB
VmSize:     2280 kB
VmLck:         0 kB
```

Figure 3.4 – Beginning of the status record for the sleep process

The `State` line contains `S (sleeping)`. This is the expected state because `sleep` is waiting for a timer rather than actively using the CPU. A sleeping process remains known to the kernel and can become runnable when the awaited event occurs.

```
root@b6f22478da98:/demo# cat /proc/$PID/status | head -20
Name:   sleep
Umask:  0022
State:  S (sleeping)
Tgid:   40
Ngid:   0
Pid:    40
PPid:   1
TracerPid:      0
Uid:    0       0       0       0
Gid:    0       0       0       0
FDSize: 256
Groups: 0
NStgid: 40
NSpid:  40
NSpgid: 40
NSsid:  1
Kthread:        0
VmPeak:     2280 kB
VmSize:     2280 kB
VmLck:         0 kB
```

Figure 3.5 – Status output showing the process in the interruptible sleeping state

Together, `ps` and `/proc` demonstrate two views of the same kernel data: `ps` formats a convenient process table, whereas `/proc/PID` exposes detailed per-process information as virtual files.

# 4 Memory Management

## 4.1 System Memory and Swap

The command `free -h` presents memory values in human-readable units. At the time of observation, the environment reported 7.7 GiB of physical memory, with approximately 4.1 GiB used, 1.3 GiB free, 2.6 GiB in buffers/cache, and 3.6 GiB available. It also reported 1.0 GiB of swap, unused at the time of measurement.

```
root@b6f22478da98:/demo# free -h
               total        used        free      shared  buff/cache   available
Mem:           7.7Gi       4.1Gi       1.3Gi       6.3Mi       2.6Gi       3.6Gi
Swap:          1.0Gi          0B       1.0Gi
```

Figure 4.1 – Human-readable physical memory and swap statistics

The `free` and `available` columns have different meanings. `free` counts memory that currently holds no data, while `available` estimates how much memory could be supplied to new applications without swapping, including reclaimable cache. Linux deliberately uses otherwise idle memory for caching, so a comparatively small `free` value does not by itself indicate a memory shortage.

Swap is disk-backed space that can hold memory pages when the kernel needs to reduce pressure on physical RAM. It is much slower than RAM and should be regarded as a fallback mechanism, not additional memory with equivalent performance. The observed zero swap usage indicates that no pages had been moved to swap at that moment.

## 4.2 Memory Used by the sleep Process

A longer `sleep 1000` process was started and its PID was saved using the shell variable `pid=$!`. Querying `VmRSS` in `/proc/$pid/status` reported 1284 kB.

```
root@b6f22478da98:/demo# sleep 1000 &
pid=$!

grep VmRSS /proc/$pid/status
[2] 48
VmRSS:      1284 kB
```

Figure 4.2 – Resident memory reported for the sleep process

`VmRSS` is the resident set size: the portion of a process's mapped memory that is currently present in physical RAM. Although `sleep` performs almost no active computation, it is still a user-space program and requires executable code, runtime libraries, stack pages, and bookkeeping data. A resident size of about 1.3 MB is therefore reasonable and remains very small relative to the available system memory.

# 5 Storage, Mounts, and Devices

## 5.1 Block Devices

The `lsblk` utility lists block devices and their relationships. The output shows several network block devices and two virtual disks. Device `vda` has a partition `vda1`, while entries associated with `vdb` are mounted at container configuration files such as `/etc/resolv.conf`, `/etc/hosts`, and `/etc/hostname`.

```
root@b6f22478da98:/demo# lsblk
NAME    MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
nbd0     43:0    0     0B  0 disk
nbd1     43:32   0     0B  0 disk
nbd2     43:64   0     0B  0 disk
nbd3     43:96   0     0B  0 disk
nbd4     43:128  0     0B  0 disk
nbd5     43:160  0     0B  0 disk
nbd6     43:192  0     0B  0 disk
nbd7     43:224  0     0B  0 disk
vda     254:0    0 926.3G  0 disk
`-vda1  254:1    0 926.3G  0 part /etc/resolv.conf
                                  /etc/hosts
                                  /etc/hostname
vdb     254:16   0 645.1M  1 disk
nbd8     43:256  0     0B  0 disk
nbd9     43:288  0     0B  0 disk
nbd10    43:320  0     0B  0 disk
nbd11    43:352  0     0B  0 disk
nbd12    43:384  0     0B  0 disk
nbd13    43:416  0     0B  0 disk
nbd14    43:448  0     0B  0 disk
nbd15    43:480  0     0B  0 disk
```

Figure 5.1 – Block devices visible from inside the Docker container

The device view is virtualized by Docker. It should not be interpreted as a direct inventory of the Mac's physical disks; it is the storage topology exposed to the Linux environment.

## 5.2 The Root Filesystem

The command `findmnt /` reports the filesystem mounted at the container root. The result identifies the source as `overlay` and the filesystem type as `overlay`.

```
root@b6f22478da98:/demo# findmnt /
TARGET SOURCE FSTYPE OPTIONS
/      overlay
              overla rw,relatime,lowerdir=/var/lib/desktop-containerd/d
```

Figure 5.2 – Mount information for the container root filesystem

OverlayFS combines multiple directory layers into one filesystem view. Docker uses this

mechanism so that read-only image layers can be shared while each container receives a writable upper layer. Paths in the mount options refer to Docker's internal storage, which in Docker Desktop ultimately resides in the Linux virtual machine backed by the macOS host. Therefore, the result does not mean that the Linux root is simply the host's `/var` directory; rather, the container root is an overlay assembled from runtime-managed layers.

### 5.3 Device Nodes

The `ls /dev` command lists the device nodes made available to the container. The result includes terminal-related nodes such as `console`, `tty`, `ptmx`, and `pts`; standard-stream links such as `stdin`, `stdout`, and `stderr`; and special character devices such as `null`, `zero`, `random`, and `urandom`.

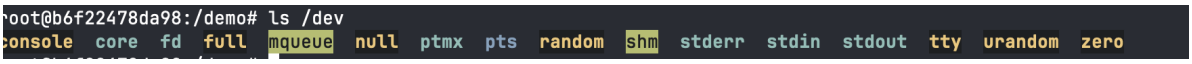



Figure 5.3 – Device nodes exposed in the container's dev directory

Device nodes provide a file-oriented interface to kernel drivers and pseudo-devices. For example, writes to `/dev/null` are discarded, reads from `/dev/zero` return zero bytes, and `/dev/urandom` supplies pseudorandom data. The limited list is another effect of container isolation: Docker exposes only the devices required or explicitly permitted for the container.

## 6 Discussion

The observations connect the principal responsibilities of an operating system to concrete Linux interfaces:

- **Files and protection:** directory listings expose metadata, and `chmod` changes access rights through permission bits.
- **Processes and CPU scheduling:** `ps`, shell job control, and `/proc` reveal process identities, parent-child relationships, and execution states.
- **Memory:** `free` summarizes system-wide allocation, while `VmRSS` reports resident memory for one process.
- **Storage and filesystems:** `lsblk` displays block devices, and `findmnt` identifies the overlay filesystem used for the container root.
- **Input and output:** `/dev` presents devices and pseudo-devices through filesystem entries.

Containerization affects every observation. The container shares the host kernel but operates with isolated namespaces and a restricted resource view. PID 1 is the container's main shell, the root filesystem is an overlay, and only selected devices are present. These differences are useful examples of how the kernel can present distinct operating-system environments to different groups of processes.

# 7 Conclusion

The laboratory work demonstrated how Linux exposes the resources managed by an operating system through a small set of command-line tools and virtual filesystems. File listings and `chmod` showed ownership and access control; job-control commands, `ps`, and `/proc` showed process creation and state; `free` and `VmRSS` described memory at system and process level; and `lsblk`, `findmnt`, and `/dev` revealed the container's storage and device interfaces.

The results also showed why the execution environment matters. Because the commands ran in Docker, they describe an isolated Linux container rather than the complete macOS host. Even so, the same core abstractions—files, processes, virtual memory, mounted filesystems, and device nodes—are the mechanisms through which Linux organizes and protects computing resources.